COS30049 Computing Technology Innovation Project

Assignment 2: Data Processing and Machine Learning

Bitcoin Address Activity Explorer

Detecting Unusual Transactions with Machine Learning

Group	HCMC_COS30049.1_4 (Group 4)
Tutor	Arthur Hoang
Cam Dan Bui	105967074
Cao Tri Tran	106214726
Le Quang Anh Hoang	106218605
Minh Khang Nguyen	106216803

Swinburne University of Technology, Ho Chi Minh City campus

Submitted 7 November 2026

Word count: 2,871 words in the body (Sections 1–8), counted by `tools/report_wordcount.py`; tables, captions, the cover, contents, references and appendix are excluded.

Contents

1 Introduction **1**

2 Problem Framing **1**

3 Data Collection **1**
3.1 Our own collections: DS1, DS2 and DS10 1
3.2 Public labelled data and text corpora 2
3.3 Challenges and how we addressed them 2

4 Data Processing **2**

5 Data Analysis **4**

6 Machine Learning Model Selection **4**

7 Implementation **6**
7.1 Technical implementation 6
7.2 Implementation evaluation 6

8 Conclusion **9**

References **10**

A Example prediction **12**

B Further results **12**

C Reproducing the results **15**

List of Figures

1 The processing pipeline. Our Bitcoin data (top) feeds the scoring methods; the labelled datasets (bottom) feed the classifiers. The two paths never mix. 3
2 DS1's four features. Amounts (F1), gaps (F3) and size ratios (F4) span orders of magnitude, which is why they are compared on log scales with robust statistics. . . 4
3 Received and sent amounts per DS1 main address (box: quartiles; line: median). Scales differ by four orders of magnitude, and many addresses send amounts unlike those they receive. 5
4 Recall on planted transactions by pattern (mean and standard deviation over five reporting seeds). No method is best at every pattern; the dashed line is Assignment 1's 70% target. 7
5 Every method, frozen as tuned on DS1, on DS2 and the held-out DS10 (left: mean planted recall with 95% bootstrap intervals on DS10; right: share of addresses breaking the sanity rule). 8
6 Elliptic test F1 per time step. Every model fails from step 43, after the shutdown of a dark market changed the illicit class's behaviour. 12
7 Unsupervised scores against the labelled datasets' own classes. Every unsupervised method is at or below chance (1×), while the supervised random forest is far above it: being unusual is not the same as belonging to a labelled class (`label_check.csv`). 13
8 Recall against the strength of each planted pattern (`eval_sensitivity.csv`). Weaker plants are harder for every method; regression keeps finding small spikes that the baseline misses. 13

9 Mean planted recall over 100 plant seeds on DS1 and DS10 and 20 on DS2 (stats_methods_summary.csv). 1
10 Every training stage's gate decision with its bootstrap interval (stages.csv). . . . 14
11 TxGPT's validation loss for three sizes and three seeds each (txgpt_sizes.csv). The sizes tie, so the smallest was kept. . . . 14
12 The Assistant's fully correct rate on each test set, before (round 1) and after (round 2) its second training round, with 95% bootstrap intervals (stats_writers_summary.csv). 15

List of Tables

1 The datasets and the role of each (tx: transactions). . . . 2
2 The four explained features (A1, refined). Each becomes a robust z-score against the same address's earlier history. . . . 3
3 Every model trained, by family. All were tuned on data that never produced a reported result. . . . 6
4 Assignment 1's objectives for the selected method (the robust z baseline) and the results. . . . 7
5 Scoring methods on DS1 (sorted by mean planted recall). . . . 7
6 Classifiers on the labelled datasets. . . . 8

1 Introduction

Banks, exchanges and payment providers that handle cryptocurrency must decide which of many transactions deserve a closer look, the topic the brief calls cybersecurity threat detection in banking. A Bitcoin address's history is public, but a block explorer shows it only as a long, unexplained list.

Our project, the Bitcoin Address Activity Explorer, ranks every transaction of an address by how *unusual* it is *for that address's own past* and says why in plain words. Its intended user is a student researcher or analyst examining one address who needs to know where to look first. It is an investigation aid, never a verdict: every flag carries the line *Unusual activity requires investigation; it does not indicate wrongdoing.*

This report covers Assignment 2: the data we collected and processed, the analysis that shaped the features, the models we trained and why we chose them, and their evaluation. Every model was trained by our team, and both language models were trained from scratch. The prediction interface that Assignment 3's web application will call is ready.

2 Problem Framing

The challenge. We must decide which transactions deserve attention when no labels exist, and explain each decision, because the user acts on the reason, not on a score.

Limits of existing solutions. Block explorers list transactions without ranking them. A fixed global rule, such as "flag anything above one bitcoin", ignores scale: the median amounts of our addresses differ by a factor of 9,949 (Section 5). Supervised models trained on the public labelled datasets learn from sparse labels and from aggregated or anonymised features that cannot be computed for an arbitrary address (Akcora et al. 2020; Elmougy and Liu 2023), and their accuracy collapses when behaviour changes over time (Pareja et al. 2020).

Why machine learning. Learning can model each address's normal behaviour, find structure without labels, read a history as a sequence, and use labels where they exist (Hilal, Gadsden and Yawney 2022; Han et al. 2022). Four research questions guide the work:

RQ1 Which stable method best finds unusual transactions planted in an address's own history?
RQ2 How well do supervised classifiers perform on labelled data over time and on unseen families?
RQ3 Does "unusual" coincide with the labelled datasets' own illicit class?
RQ4 Does a sequence model trained from scratch on other addresses generalise better than per-address statistics, and can a language model trained from scratch explain flags faithfully?

3 Data Collection

Table 1 lists the eight datasets: three collections of public blockchain data of our own, two public labelled datasets, and three text corpora for the language models.

3.1 Our own collections: DS1, DS2 and DS10

Assignment 1 chose Blockstream's Esplora API (Blockstream n.d.) because it returns each address's confirmed history with every transaction's inputs and outputs. The DS1 selection rule was recorded, with its random seed, before any address was fetched: take 20 block heights from the previous four weeks, gather their output addresses (5,995 candidates), shuffle them and visit them in order. A *main* address needs 50–1,000 confirmed transactions over at least 90 days; a *short-history* address needs 3–9, so the Explorer's "not enough history" state is tested too. An address is kept only if its transaction count matches the API's own count, its received and sent sums match the API's totals, and no transaction is duplicated or unconfirmed. DS1 holds 25 main and 3 short addresses with 7,966 transactions.

Table 1: The datasets and the role of each (tx: transactions).

ID	Dataset	Source (licence)	Size	Role
DS1	Our Esplora snapshot	Blockstream Esplora API (public data)	25 + 3 addresses, 7,966 tx	Every scoring method; evaluation
DS2	Our Esplora crawl (hashed IDs)	Esplora API via mempool.space	4,012 addresses, 898,490 tx	Trains TxGPT; check on unseen addresses
DS3	Elliptic data set	Kaggle (CC BY-NC-ND 4.0)	203,769 tx	Classification
DS4	BitcoinHeist	UCI repository 526 (CC BY 4.0)	2,916,697 address-days	Classification
DS5	FineWeb-Edu	Hugging Face (ODC-By 1.0)	5.0 billion tokens used	Assistant pretraining
DS6	smol-smoltalk	Hugging Face (Apache 2.0)	chat conversations	Assistant chat stage
DS7	Our explanation pairs	Built from DS1 and DS2 facts	question–answer pairs	Teaching and testing the Assistant
DS10	A teammate's snapshot	Esplora API, refetched and cross-checked	27 addresses, 5,785 tx	Held-out final test of every model

DS2 applies the same client to 200 blocks spread over twelve months, after removing every address in DS1's candidate pool (1,533) so that DS1 never leaks into TxGPT's training data. Of 9,079 addresses visited, 4,012 were kept; they are stored only as salted hashes. DS10 is a teammate's independent snapshot, collected with their own script and rule. We refetched it with our client: all 5,785 transactions agree with the teammate's file, and no address is shared with DS1 or DS2.

3.2 Public labelled data and text corpora

Elliptic (DS3) labels 46,564 of its 203,769 transactions as illicit or licit, its own class names, over 49 time steps (Elmougy and Liu 2023). BitcoinHeist (DS4) has 2,916,697 address-days, of which 41,413 belong to ransomware families (Akcora et al. 2020; UCI Machine Learning Repository 2020). Elliptic's licence allows no derivatives, so we ship a download script instead of processed data. The Assistant learned English from FineWeb-Edu (DS5; Penedo et al. 2024), conversation from smol-smoltalk (DS6; Ben Allal et al. 2025), and explanations from DS7, question–answer pairs built from our own pipeline's facts.

3.3 Challenges and how we addressed them

- **Rate limits.** The public API allows about 700 requests an hour. Our client paces requests, backs off on refusals, falls back to the compatible mempool.space API (mempool.space n.d.), caches every raw reply and resumes after interruptions; 106 addresses that failed every retry were logged and skipped.
- **Selection bias.** 539 visited addresses exceeded the transaction cap (typically busy services) and 4,422 had too little history. Sampling recent blocks favours active addresses, and an address is not a person; we state these limits rather than hide them.
- **No ground truth.** Real transactions carry no labels, so we evaluate on transactions we plant (Section 4) and use labelled metrics only on DS3 and DS4.

4 Data Processing

Figure 1 shows the pipeline; each step is one module of our `explorer` package.

Cleaning and missing values. DS1 passes five checks: parse every saved page, keep confirmed transactions, remove duplicate (address, transaction) pairs, drop transactions that neither pay nor

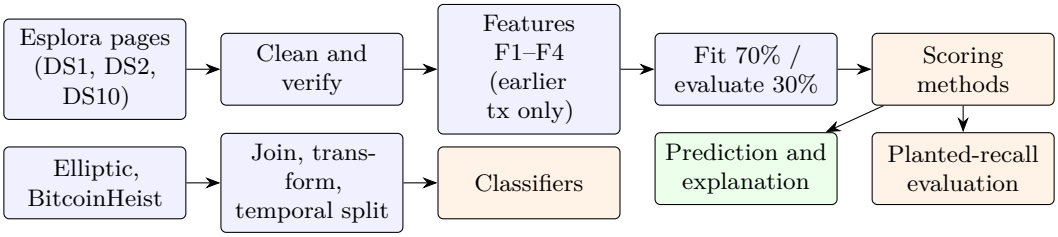


Figure 1: The processing pipeline. Our Bitcoin data (top) feeds the scoring methods; the labelled datasets (bottom) feed the classifiers. The two paths never mix.

spend from the address, and sort by time. None removed a row, because the collection checks had already rejected faulty histories. The source data has no missing values; features are empty by design when an address has too little history, and such transactions are shown as "not scored" rather than imputed.

Feature engineering. For each transaction we compute the address's net amount (received minus sent) and direction, then the four features of Table 2. Each is compared with the same address's history using a robust z-score: the distance from the median in units of the median absolute deviation, which one extreme past value cannot distort (Blázquez-García et al. 2022). Heavy-tailed features are compared on a logarithmic scale, and the spread has a floor so that a run of identical values cannot divide by zero. This is the normalisation: every score is relative to the address itself.

Table 2: The four explained features (A1, refined). Each becomes a robust z-score against the same address's earlier history.

ID	Name	Definition	Compared on
F1	Size	Amount the address received or sent (BTC)	log10; same direction only
F2	Burst	Earlier transactions in the previous 7 days	raw count
F3	Gap	Hours since the address's previous transaction	log10
F4	Relative size	Amount divided by the usual amount in the same direction	log10

Each feature is compared with earlier transactions of the same address in the previous 30 days, extended back to reach 20; a transaction is scored once the address has at least 10 earlier transactions over 14 days.

Time safety. A transaction's features use only *strictly earlier* transactions of the same address, and transactions in the same block are never each other's history. Models fit on each address's earlier 70% and are evaluated on the rest, and every scaler sits inside a scikit-learn pipeline fitted on training rows only. These rules prevent leakage (Kapoor and Narayanan 2023).

Planted transactions. To measure detection without labels we plant three patterns into copies of each history's evaluation period (Lai et al. 2021): a *spike* of 10× the address's usual amount, a *burst* of 10 transactions within 24 hours, and a *dormant* address waking after 60–120 days. Each seed plants 50 of each pattern. Tuning seeds never produce reported results.

Labelled datasets. Elliptic's features and classes are joined on the transaction identifier and split by time (steps 1–34 to train and validate, 35–49 to test), with all features (AF) or local ones (LF). In BitcoinHeist every ransomware family becomes the positive class, the five heavy-tailed features are log transformed, and `year`, `day` and `address` are excluded as features because they would leak the split or identify the row.

Language-model inputs. TxGPT writes each transaction as six tokens (gap, direction, amount, inputs, outputs, fee rate) from a vocabulary of 94. DS7 pairs a question about one transaction with its facts and the template's answer, split by address so that no address spans two splits.

5 Data Analysis

The analysis lives in `notebooks/01_eda.ipynb`, and every statistic below is written to `eda_tests.csv`. Each finding is stated as an observation, the statistic behind it, and the decision it informed.

Finding 1: the features are heavy-tailed. Amounts, gaps and ratios span several orders of magnitude (Figure 2). The raw amount has a skewness of 38.9, but only -0.44 on a logarithmic scale; the size ratio's excess kurtosis is 7,813. A mean and standard deviation would be dominated by a few extreme values, so we compare on log scales and use the median and median absolute deviation.

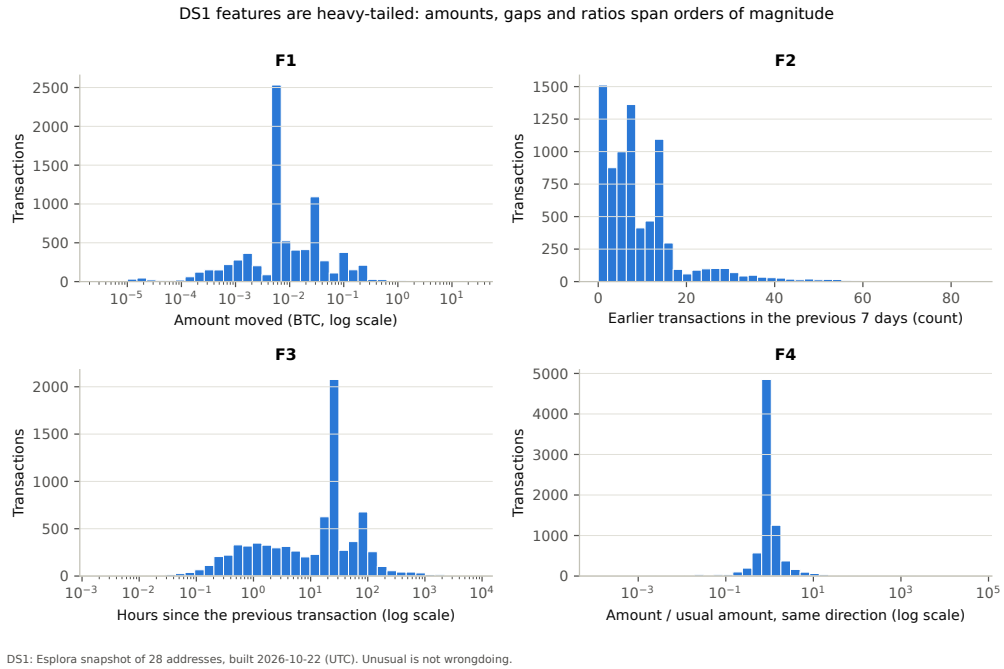


Figure 2: DS1's four features. Amounts (F1), gaps (F3) and size ratios (F4) span orders of magnitude, which is why they are compared on log scales with robust statistics.

Finding 2: every address has its own scale. The median amounts of DS1's main addresses differ by a factor of 9,949, and for 9 of the 23 addresses with enough transactions in both directions, received and sent amounts differ (Mann–Whitney U, $p < 0.05$; Figure 3). One global threshold cannot fit every address, so each feature is compared with the same address's history and amounts only with earlier transactions in the same direction. The ablation confirms the decision: A1's original definition, without these refinements, flags 31.3% of real transactions with a mean planted recall of 0.439, against 10.4% and 0.655 for the refined features (`eval_ablation.csv`).

Finding 3: labels are rare and drift over time. Elliptic's illicit share among labelled transactions is 11.6% in the training steps but 6.5% in the test steps, and 77% of all rows are unlabelled. BitcoinHeist's positives are 1.4% of rows, and every feature separates them only weakly (largest rank-biserial effect 0.13). Accuracy would reward predicting the majority class (Chicco and Jurman 2020), so we split by time, report precision, recall, F1 and the area under the precision–recall curve (PR-AUC), and include class weights in tuning. The weak BitcoinHeist effects also anticipated its poor classification results in Section 7.2.

6 Machine Learning Model Selection

Criteria. A scoring method for the Explorer had to (1) work without labels, (2) judge each transaction against its own address's earlier history, (3) give a reason the user can check, (4) be

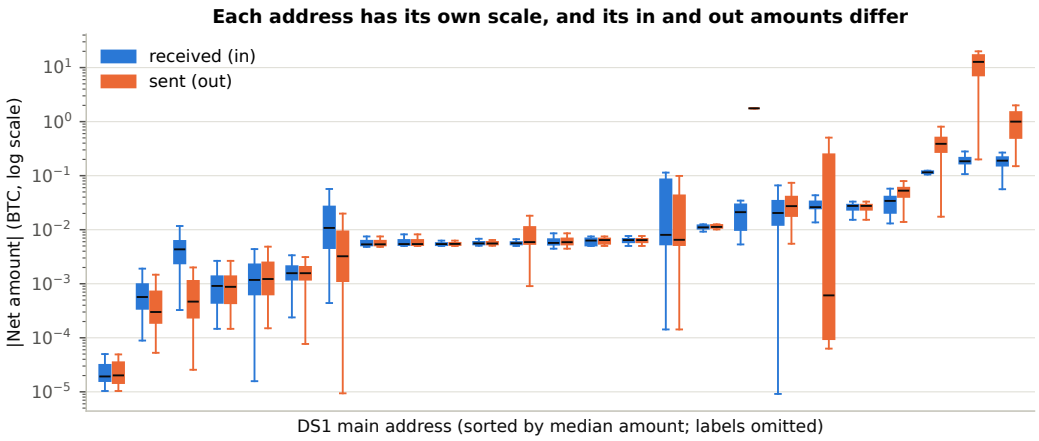


Figure 3: Received and sent amounts per DS1 main address (box: quartiles; line: median). Scales differ by four orders of magnitude, and many addresses send amounts unlike those they receive.

stable, keeping a top-50 overlap (Jaccard) of at least 0.7 across refits, (5) flag no more than 20% of any address's transactions, since an address flagged everywhere is simply different, and (6) run on a laptop CPU. The brief adds that at least two of classification, clustering and regression are used.

Candidates (Table 3). No detector wins on every kind of anomaly (Han et al. 2022), so each family covers a different idea of "unusual". The robust z baseline flags a transaction when any feature lies far from the address's median. Clustering learns the shape of normal transactions and scores the distance to it (Ikotun et al. 2023; Bushra and Yi 2021). Regression predicts each amount and gap from earlier history and scores the miss; conformal calibration turns its quantile models into ranges with stated coverage (Angelopoulos and Bates 2023). Isolation Forest and the Local Outlier Factor (LOF) are general outlier detectors (Boukerche, Zheng and Alfandi 2021). TxGPT, a small transformer trained from scratch on DS2, reads a history as a sequence and scores each transaction by its surprisal, the negative log-probability of what happened, as sequence models do for event logs (Le and Zhang 2022). For labelled data we compared a linear model, a neighbour-based model and two tree ensembles, which remain strong on tabular data (Grinsztajn, Oyallon and Varoquaux 2022).

Tuning. Every grid is in the settings file and every search in `tuning_*.csv`, run on plant seeds that never score reported results. After the first round, each method was retrained in stages, and a stage was kept only if it passed a gate fixed in advance: higher recall with a bootstrap interval above zero, and no pattern falling by more than its seed spread. LOF, K-Means and DBSCAN passed (`stages.csv`).

The selection rule, fixed in Assignment 1, picks the stable method with the highest mean planted recall, and within five points the easier one to explain. Fusion scored highest (0.682), but the baseline (0.655) lies within the margin, and the difference is not significant (paired bootstrap, $p = 0.324$). The Explorer therefore uses the baseline, whose reasons map directly to the features.

Explanations. The *Explorer Assistant*, a language model trained from scratch, may phrase each explanation, but only if its answer passes a code guardrail: every number from the facts, the limitation line present, no banned word, comparison words pointing the right way, and a verdict matching the flag. Otherwise a fixed template is shown, because language models can state numbers absent from their input (Ji et al. 2023).

Table 3: Every model trained, by family. All were tuned on data that never produced a reported result.

Family	Models	Data	Output	Tuned
Statistics	Robust z-score baseline (A1)	DS1	Largest $\|z\|$ over F1–F4	Flag threshold
Clustering	K-Means; DBSCAN	DS1	Distance to normal clusters	k; eps, min. samples
Clustering	K-Means address profiles	DS1, DS2	Activity profile in words	k (silhouette)
Regression	Ridge; random forest; hist. gradient boosting	DS1	Miss on predicted amount and gap	Per-model grids
Regression	Quantile boosting + conformal	DS1	Distance outside a calibrated range	Quantiles, coverage
Outlier detection	Isolation Forest; LOF (novelty)	DS1	Isolation depth; local density	Trees, samples; neighbours
Combination	Score fusion	DS1	Weighted pair of scores	Weights at a flag budget
Classification	Logistic regression; KNN; random forest; hist. gradient boosting	DS3, DS4	Class probability	Validation split
Sequence model	TxGPT (trained from scratch)	DS2	Surprisal of each transaction	3 sizes × 3 seeds
Language model	Explorer Assistant, 152M (trained from scratch)	DS5–DS7	Explanation (guardrailed)	Adoption rule

7 Implementation

7.1 Technical implementation

The project is written in Python with pandas, scikit-learn and matplotlib, as the brief asks (The pandas development team 2024; scikit-learn developers 2025; The Matplotlib Development Team 2025); PyTorch is used only for the two language models (Ansel et al. 2024). The `explorer` package has one module per pipeline step, all run through one command-line tool; `config/settings.json` holds every threshold, window, grid and seed, so the code contains no unexplained constants. Versions are pinned in a conda environment, and a test suite of 266 tests checks properties such as time safety and the guardrail. A full rebuild of every DS1 result runs offline on a laptop CPU, and a script compares the rebuilt tables with the committed ones number by number (Pineau et al. 2021).

TxGPT and the Assistant were trained on GPUs. TxGPT was trained in three sizes with three seeds each; the sizes tie, so the smallest (820,352 parameters) was kept. The Assistant follows a modern decoder design (Touvron et al. 2023) and was pretrained on 5.0 billion DS5 tokens, more than the compute-optimal budget of about 20 tokens per parameter (Hoffmann et al. 2022), then taught conversation (DS6) and explanations (DS7). It runs on a CPU at prediction time.

The `Predictor` class is the interface Assignment 3 will call: it scores an address's whole timeline, scores a hypothetical next transaction (the brief's "user input to prediction"), and explains any transaction.

7.2 Implementation evaluation

Objectives. Table 4 compares the results with the targets Assignment 1 set before any data existed. The baseline meets the recall target on spikes and dormant addresses but not on bursts, and four DS1 addresses break the sanity rule. We report both rather than move the targets.

Scoring methods on DS1 (Table 5). Each method finds different patterns (Figure 4). Regression finds every planted burst event but few dormant addresses, LOF misses most spikes, and only fusion and the baseline catch most plants of all three patterns. TxGPT flags the fewest real transactions (1.5%) and so has the highest precision lower bound (0.421 against 0.125): a flag that is not a plant

Table 4: Assignment 1's objectives for the selected method (the robust z baseline) and the results.

Objective (set in Assignment 1)	Result	Status
Snapshot of 20–30 main addresses (50+ tx, 90+ days) and 2–3 short ones	25 main and 3 short addresses	Met
Planted recall $\geq$ 70% for 2 of 3 patterns	Spike 0.744, dormant 0.840, burst 0.381	Met
Stability: top-50 Jaccard $\geq$ 0.7	1.000	Met
Sanity: no address over 20% flagged	DS1: 4 addresses over; DS10: 0	Partly met
Method agreement reported	Baseline–fusion Jaccard 0.512	Met
Blind reviewer check (Cohen's kappa)	Pending: two teammates rate reports/reviewer/sheet.csv	Open
Accuracy, precision, recall, F1 where labels exist	DS3 and DS4 (Table 6)	Met

may still be worth a look, so precision on real data can only be bounded, never measured.

Table 5: Scoring methods on DS1 (sorted by mean planted recall).

Method	Family	Spike	Burst	Dormant	Mean [95% CI]	Stab.	Flags
Score fusion	Combination	0.780	0.498	0.768	0.682 [0.573, 0.781]	0.736	9.4%
Robust z baseline (A1) ⋆	Statistics	0.744	0.381	0.840	0.655 [0.546, 0.753]	1.000	10.4%
Regression residual	Regression	0.864	0.695	0.320	0.626 [0.526, 0.722]	0.736	17.5%
TxGPT surprisal	Sequence model	0.736	0.170	0.736	0.547 [0.442, 0.638]	1.000	1.5%
DBSCAN distance	Clustering	0.544	0.109	0.776	0.476 [0.366, 0.571]	0.912	3.9%
K-Means distance	Clustering	0.624	0.050	0.736	0.470 [0.359, 0.567]	1.000	3.8%
Isolation Forest	Outlier detection	0.600	0.122	0.564	0.429 [0.321, 0.534]	0.876	3.6%
Local Outlier Factor	Outlier detection	0.116	0.066	0.812	0.331 [0.262, 0.392]	0.846	4.5%
Regression range (conformal)	Regression	0.712	0.174	0.044	0.310 [0.246, 0.373]	0.514	4.8%

Recall on planted transactions over five reporting seeds; intervals are 95% address-level bootstrap. Mean recall averages spike, burst (per transaction and per event) and dormant. Stability is the top-50 Jaccard over refits. Flag rate is the share of real evaluation-period transactions flagged. ⋆ the selected method.

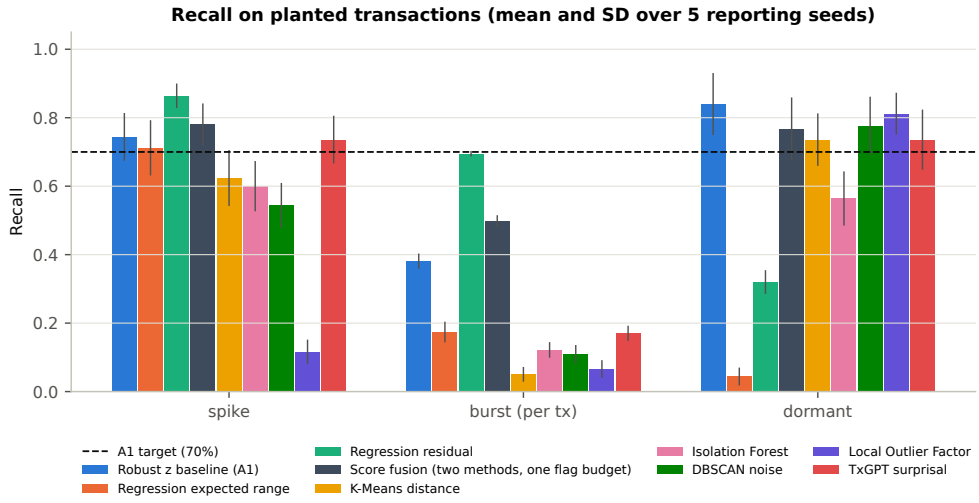


Figure 4: Recall on planted transactions by pattern (mean and standard deviation over five reporting seeds). No method is best at every pattern; the dashed line is Assignment 1's 70% target.

Unseen addresses: DS2 and DS10. The frozen methods were tested on 2,933 DS2 histories and once on DS10 (Figure 5). Two outcomes were unexpected. First, methods tuned on DS1 fall the most: fusion drops from 0.682 to 0.367 on DS10, below the baseline's 0.431. Second, TxGPT,

which learned from other addresses, ranks first on DS10 (0.598, interval [0.534, 0.657]). Over 100 plant seeds the baseline ranks first among the pooled methods on DS10 in 96% of seeds. Bursts are hard to find on DS10, whose addresses are busier and less regular.

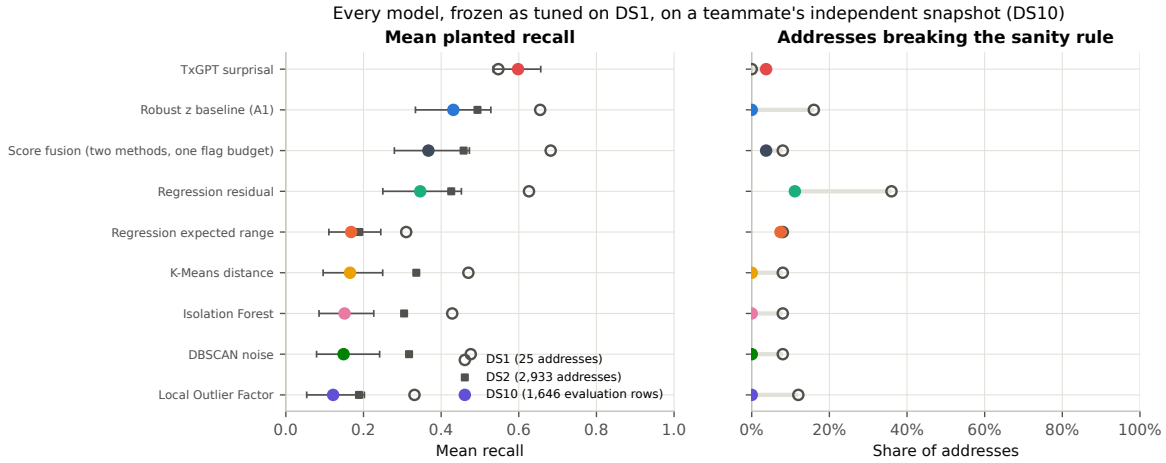


Figure 5: Every method, frozen as tuned on DS1, on DS2 and the held-out DS10 (left: mean planted recall with 95% bootstrap intervals on DS10; right: share of addresses breaking the sanity rule).

Classifiers on labelled data (Table 6). On Elliptic the random forest has the best F1 (0.815, interval [0.796, 0.835] over ten training seeds), but every model collapses after time step 43 (Figure 6, Appendix B), when a dark market closed and illicit behaviour changed (Pareja et al. 2020). On BitcoinHeist's temporal split no model's F1 exceeds 0.07, and families held out of training are found at recalls of 0.3%–3.3% by the tree models, consistent with Finding 3.

Table 6: Classifiers on the labelled datasets.

Data and split	Model	Accuracy	Precision	Recall	F1	PR-AUC
Elliptic, all features	Hist. gradient boosting	0.971	0.808	0.734	0.769	0.803
Elliptic, all features	k-nearest neighbours	0.957	0.684	0.620	0.650	0.659
Elliptic, all features	Logistic regression	0.949	0.619	0.578	0.598	0.412
Elliptic, all features	Random forest	0.979	0.928	0.727	0.815	0.792
Elliptic, local features	Hist. gradient boosting	0.965	0.738	0.721	0.730	0.784
Elliptic, local features	k-nearest neighbours	0.969	0.792	0.703	0.745	0.741
Elliptic, local features	Logistic regression	0.969	0.839	0.640	0.726	0.619
Elliptic, local features	Random forest	0.970	0.801	0.721	0.759	0.781
BitcoinHeist, temporal	Hist. gradient boosting	0.705	0.012	0.722	0.024	0.042
BitcoinHeist, temporal	k-nearest neighbours	0.789	0.011	0.452	0.021	0.025
BitcoinHeist, temporal	Logistic regression	0.170	0.006	0.976	0.012	0.005
BitcoinHeist, temporal	Random forest	0.942	0.038	0.436	0.070	0.052
BitcoinHeist, random	Hist. gradient boosting	0.975	0.199	0.252	0.222	0.151
BitcoinHeist, random	k-nearest neighbours	0.957	0.099	0.254	0.143	0.071
BitcoinHeist, random	Logistic regression	0.510	0.020	0.691	0.039	0.017
BitcoinHeist, random	Random forest	0.979	0.250	0.239	0.244	0.177

Test splits: Elliptic time steps 35–49; BitcoinHeist years after the training years (temporal) or a random 20%. Metrics are for the datasets' own positive classes (illicit; ransomware), at the threshold chosen on validation data.

Unusual is not a label. Scored against the labelled datasets' own classes, the unsupervised methods do worse than chance (PR-AUC below the class share), while the supervised random forest reaches 12.2× chance on Elliptic (Appendix B). Being unusual for an address and belonging to a labelled class are different things, which supports the Explorer's wording.

Language models. The Assistant passes the guardrail on 100.0% of DS7 test prompts and 100.0%

of DS10 prompts, and is fully correct (right reasons, order and numbers) on 96.9% and 93.1%; pooled over twelve test sets it is fully correct on 94.8% (95% interval [94.1%, 95.4%]). Its weakest area is open glossary questions, where it states 48% of the key facts.

Limitations. Plants are our guess at what unusual looks like; DS1 has only 28 addresses, so intervals are wide; the methods tuned on DS1 transfer less well; and a language model can still give a wrong reason that the guardrail does not catch, which is why the template remains the default.

8 Conclusion

We built and evaluated a pipeline that ranks a Bitcoin address's transactions by how unusual each is for that address. Our answers:

RQ1 The robust z-score baseline is the Explorer's method: it is within the margin of the best method on DS1, stable, explainable, and the most consistent on unseen addresses.

RQ2 Supervised classifiers do well on Elliptic until behaviour changes, and stay weak on BitcoinHeist across time and on unseen families.

RQ3 Unusual activity and the labelled illicit class barely overlap, so a flag is a reason to look, not a verdict.

RQ4 TxGPT, trained only on other addresses, ranks first on the held-out DS10; the Assistant explains flags faithfully under a guardrail.

The most significant insight is that methods tuned on one sample of addresses lose much of their advantage on another, so an independent held-out test was essential. Future work for Assignment 3 includes showing TxGPT's surprisal beside the baseline, more addresses, graph models for labelled data, and a usability study with the intended users.

References

[1] Akcora, C.G., Li, Y., Gel, Y.R. and Kantarcioglu, M. (2020) 'BitcoinHeist: topological data analysis for ransomware prediction on the Bitcoin blockchain', in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*, pp. 4439–4445. doi: 10.24963/ijcai.2020/612.

[2] Angelopoulos, A.N. and Bates, S. (2023) 'Conformal prediction: a gentle introduction', *Foundations and Trends in Machine Learning*, 16(4), pp. 494–591. doi: 10.1561/2200000101.

[3] Ansel, J. et al. (2024) 'PyTorch 2: faster machine learning through dynamic Python bytecode transformation and graph compilation', in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, pp. 929–947. doi: 10.1145/3620665.3640366.

[4] Ben Allal, L. et al. (2025) 'SmolLM2: when smol goes big — data-centric training of a small language model', *arXiv preprint*, arXiv:2502.02737.

[5] Blázquez-García, A., Conde, A., Mori, U. and Lozano, J.A. (2022) 'A review on outlier/anomaly detection in time series data', *ACM Computing Surveys*, 54(3), pp. 1–33. doi: 10.1145/3444690.

[6] Blockstream (n.d.) *Esplora HTTP API*. Available at: https://github.com/Blockstream/esplora/blob/master/API.md (Accessed: 22 October 2026).

[7] Boukerche, A., Zheng, L. and Alfandi, O. (2021) 'Outlier detection: methods, models, and classification', *ACM Computing Surveys*, 53(3), pp. 1–37. doi: 10.1145/3381028.

[8] Bushra, A.A. and Yi, G. (2021) 'Comparative analysis review of pioneering DBSCAN and successive density-based clustering algorithms', *IEEE Access*, 9, pp. 87918–87935. doi: 10.1109/ACCESS.2021.3089036.

[9] Chicco, D. and Jurman, G. (2020) 'The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation', *BMC Genomics*, 21, 6. doi: 10.1186/s12864-019-6413-7.

[10] Elmougy, Y. and Liu, L. (2023) 'Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics', in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, pp. 3979–3990. doi: 10.1145/3580305.3599803.

[11] Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022) 'Why do tree-based models still outperform deep learning on typical tabular data?', *Advances in Neural Information Processing Systems*, 35, Datasets and Benchmarks Track. arXiv:2207.08815.

[12] Han, S., Hu, X., Huang, H., Jiang, M. and Zhao, Y. (2022) 'ADBench: anomaly detection benchmark', *Advances in Neural Information Processing Systems*, 35, Datasets and Benchmarks Track. arXiv:2206.09426.

[13] Hilal, W., Gadsden, S.A. and Yawney, J. (2022) 'Financial fraud: a review of anomaly detection techniques and recent advances', *Expert Systems with Applications*, 193, 116429. doi: 10.1016/j.eswa.2021.116429.

[14] Hoffmann, J. et al. (2022) 'An empirical analysis of compute-optimal large language model training', *Advances in Neural Information Processing Systems*, 35. arXiv:2203.15556.

[15] Ikotun, A.M., Ezugwu, A.E., Abualigah, L., Abuhaija, B. and Heming, J. (2023) 'K-means clustering algorithms: a comprehensive review, variants analysis, and advances in the era of big data', *Information Sciences*, 622, pp. 178–210. doi: 10.1016/j.ins.2022.11.139.

[16] Ji, Z. et al. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), pp. 1–38. doi: 10.1145/3571730.

[17] Kapoor, S. and Narayanan, A. (2023) 'Leakage and the reproducibility crisis in machine-learning-based science', *Patterns*, 4(9), 100804. doi: 10.1016/j.patter.2023.100804.

[18] Lai, K.-H., Zha, D., Xu, J., Zhao, Y., Wang, G. and Hu, X. (2021) 'Revisiting time series outlier detection: definitions and benchmarks', *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1 (NeurIPS 2021). Available at: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/ec5decca5ed3d6b8079e2e7e7bacc9f2-Abstract-round1.html (Accessed: 29 October 2026).

[19] Le, V.-H. and Zhang, H. (2022) 'Log-based anomaly detection with deep learning: how far are we?', in

Proceedings of the 44th International Conference on Software Engineering (ICSE '22), pp. 1356–1367. doi: 10.1145/3510003.3510155.

[20] mempool.space (n.d.) *REST API documentation*. Available at: https://mempool.space/docs/api/rest (Accessed: 22 October 2026).

[21] Pareja, A. et al. (2020) 'EvolveGCN: evolving graph convolutional networks for dynamic graphs', *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04), pp. 5363–5370. doi: 10.1609/aaai.v34i04.5984.

[22] Penedo, G. et al. (2024) 'The FineWeb datasets: decanting the web for the finest text data at scale', *Advances in Neural Information Processing Systems*, 37, pp. 30811–30849. doi: 10.52202/079017-0970.

[23] Pineau, J. et al. (2021) 'Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)', *Journal of Machine Learning Research*, 22(164), pp. 1–20.

[24] scikit-learn developers (2025) *scikit-learn* (Version 1.6.1) [Software]. Zenodo. doi: 10.5281/zenodo.14627164.

[25] The Matplotlib Development Team (2025) *Matplotlib: visualization with Python* (Version 3.10.1) [Software]. Zenodo. doi: 10.5281/zenodo.14940554.

[26] The pandas development team (2024) *pandas-dev/pandas: Pandas* (Version 2.2.3) [Software]. Zenodo. doi: 10.5281/zenodo.13819579.

[27] Touvron, H. et al. (2023) 'LLaMA: open and efficient foundation language models', *arXiv preprint*, arXiv:2302.13971.

[28] UCI Machine Learning Repository (2020) *Bitcoin Heist Ransomware Address* [Dataset]. doi: 10.24432/C5BG8V. Available at: https://archive.ics.uci.edu/dataset/526/bitcoinheistransomwareaddressdataset (Accessed: 20 October 2026).

A Example prediction

The output below is what Assignment 3's back end receives. It was produced by `python -m explorer.cli predict what-if` on a hypothetical transaction of one DS1 address: sending 0.05 BTC, about a hundred times the address's usual amount. The size features (F1, F4) pass the flag line; the timing features (F2, F3) are within the usual range, so they are not mentioned.

```
input:    0.05 BTC out at 2026-10-20T12:00Z, address 1Bbcsfxj2JRUD5crcdZVHBcViT61vrJ3dz
scored: true   flagged: true   score (largest |z|): 5.98
feature                    value        usual       z  role
F1 Size                     0.05  0.000524948    5.88  flag
F2 Burst                       3            4   -0.45  within-usual
F3 Gap                   21.8264      10.0535    0.30  within-usual
F4 Relative size         95.2472     0.909938    5.98  flag
```

Explanation shown to the user (template writer):

```
Flagged by Robust z baseline (A1).
What stands out compared with this address's own history:
- Higher ratio to the usual amount than usual: 95.2x the usual amount for this direction, against a
  usual 0.9x.
- Larger amount than usual: 0.05 BTC moved, against a usual 0.00052495 BTC.
Unusual activity requires investigation; it does not indicate wrongdoing.
```

B Further results

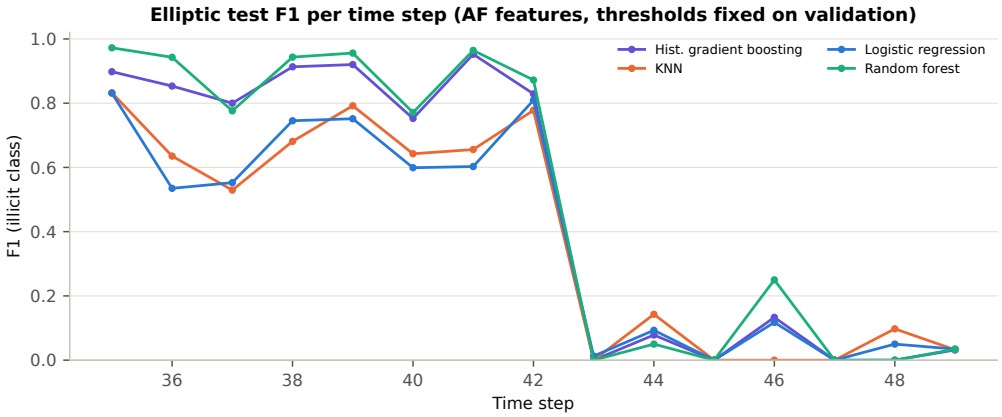


Figure 6: Elliptic test F1 per time step. Every model fails from step 43, after the shutdown of a dark market changed the illicit class's behaviour.

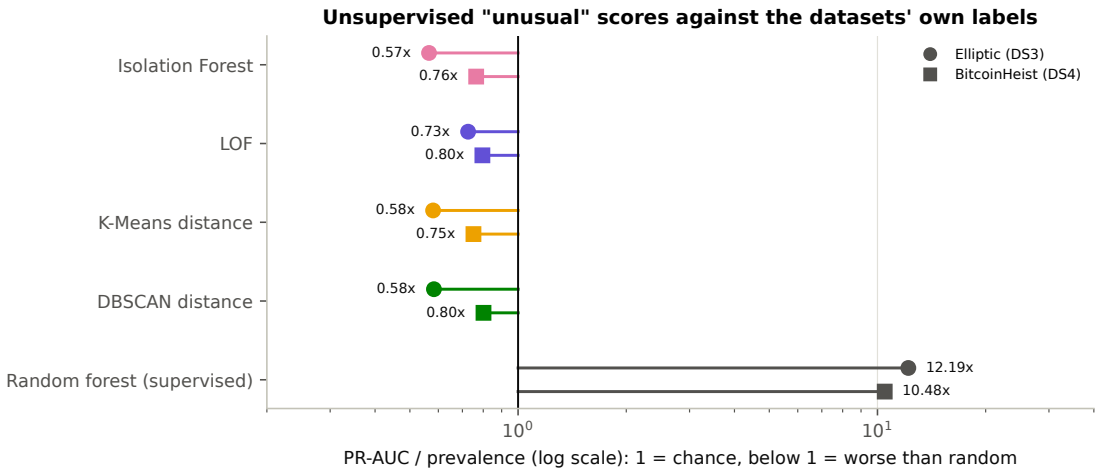


Figure 7: Unsupervised scores against the labelled datasets' own classes. Every unsupervised method is at or below chance (1×), while the supervised random forest is far above it: being unusual is not the same as belonging to a labelled class (`label_check.csv`).

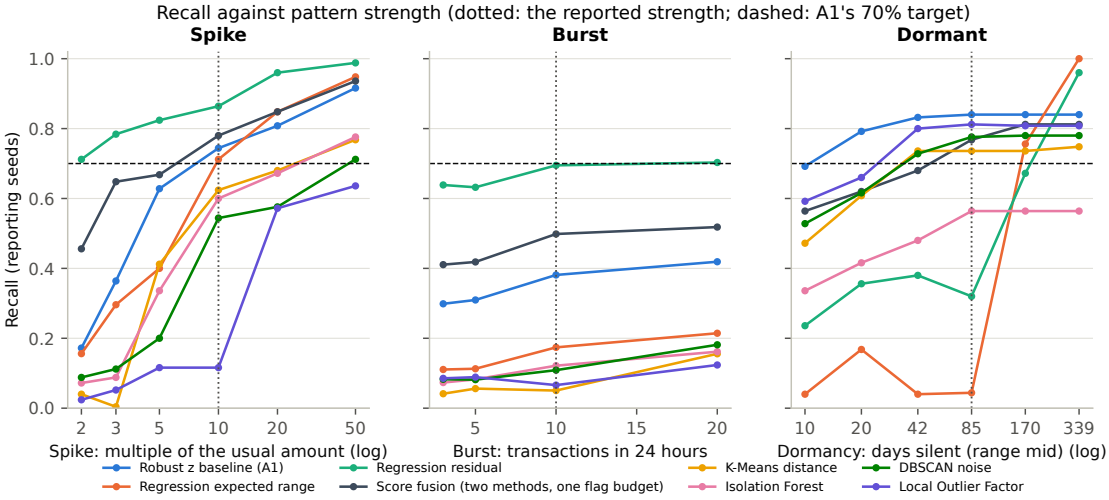


Figure 8: Recall against the strength of each planted pattern (`eval_sensitivity.csv`). Weaker plants are harder for every method; regression keeps finding small spikes that the baseline misses.

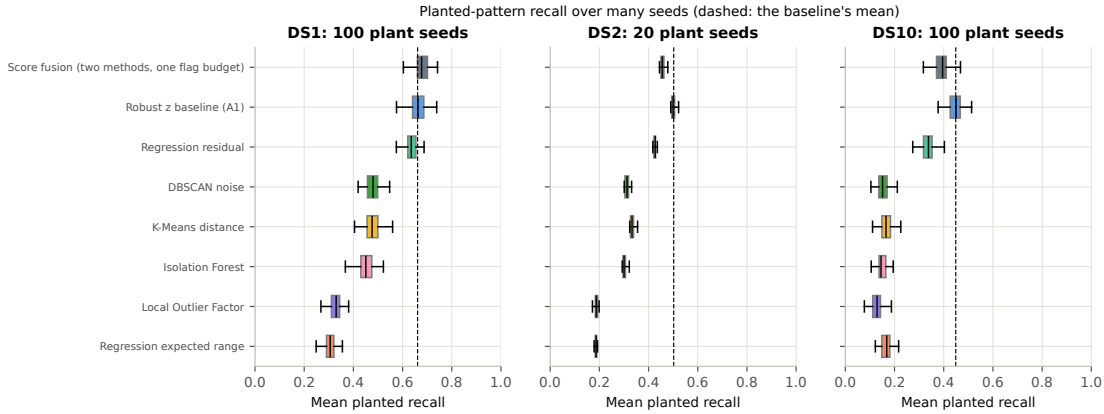


Figure 9: Mean planted recall over 100 plant seeds on DS1 and DS10 and 20 on DS2 (`stats_methods_summary.csv`).

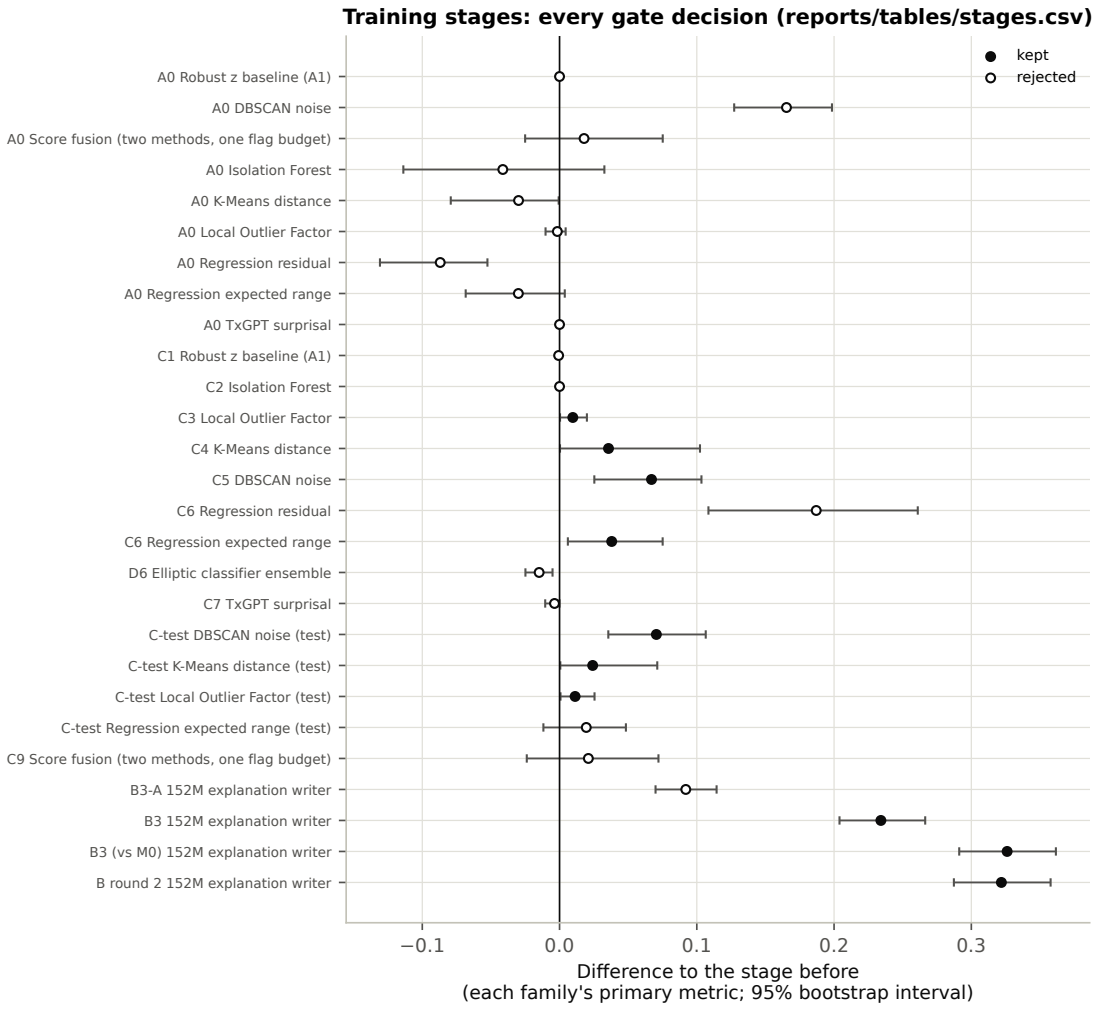


Figure 10: Every training stage's gate decision with its bootstrap interval (`stages.csv`).

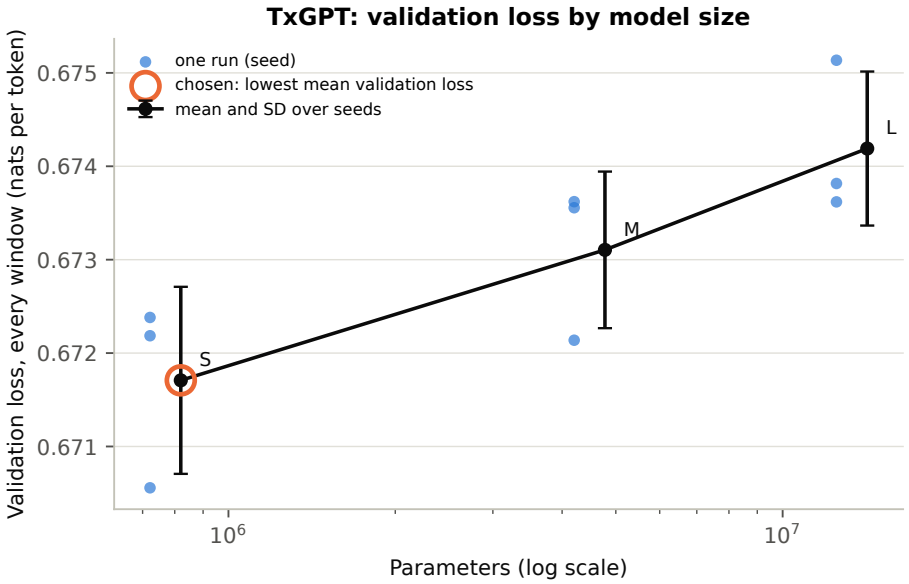


Figure 11: TxGPT's validation loss for three sizes and three seeds each (`txgpt_sizes.csv`). The sizes tie, so the smallest was kept.

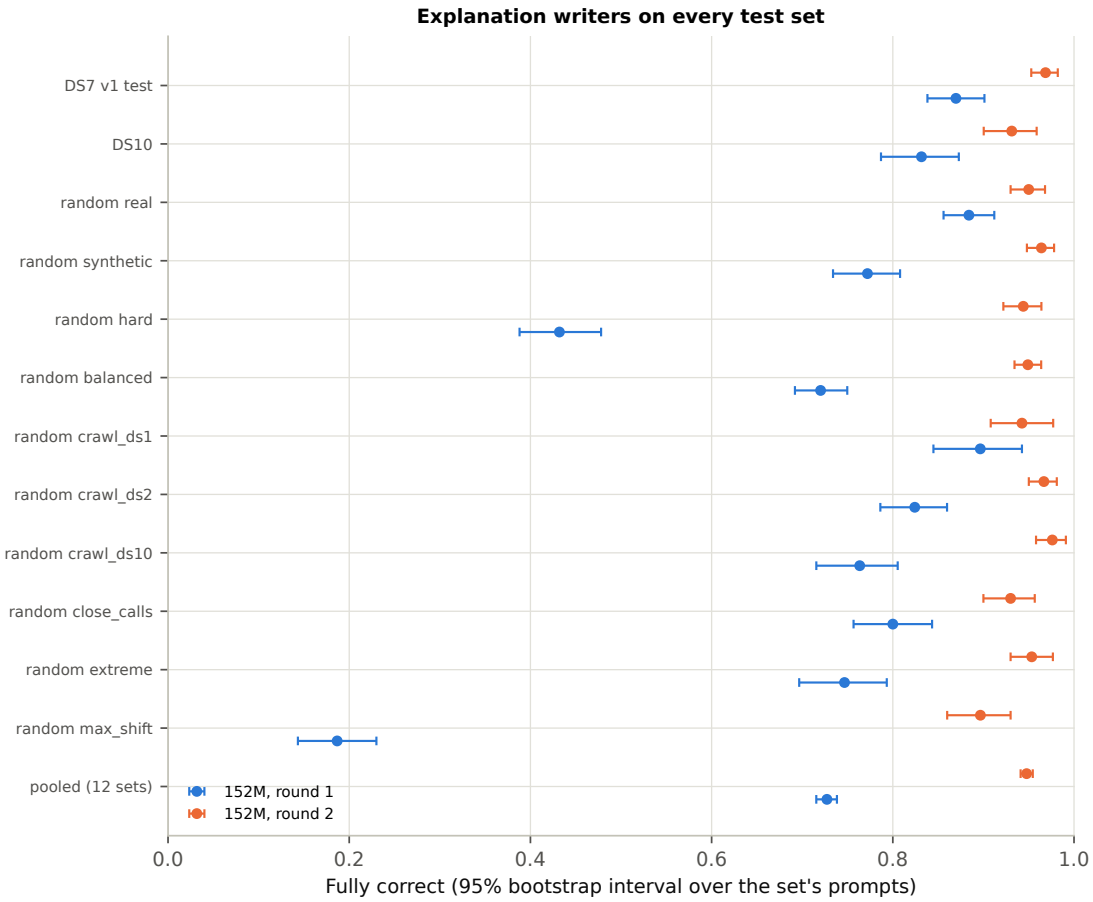


Figure 12: The Assistant's fully correct rate on each test set, before (round 1) and after (round 2) its second training round, with 95% bootstrap intervals (`stats_writers_summary.csv`).

C Reproducing the results

The repository's `README.md` gives every command. In short:

```
conda env create -f environment.yml
conda activate cos30049-a2
python -m pytest -q                          # the test suite
python -m explorer.cli all --offline         # every DS1 result from the shipped data
python -m explorer.cli figures
python tools/compare_tables.py               # compare the rebuild with the committed tables
```

`DATA_CARD.md` documents every dataset, file and column, and `MODEL_CARD.md` every final model.